

LeaveApp - Application Structure

Overview

LeaveApp is a full-stack leave management system built with Flask (backend) and React (frontend). It supports employee leave requests, admin approval workflows, leave balance tracking, and gender-based leave restrictions.

Tech Stack

Layer	Technology
Backend	Python 3.14, Flask 3.0, Flask-SQLAlchemy, Flask-JWT-Extended
Frontend	React 18, Vite 5, React Router 6, Axios
Database	SQLite (development) / PostgreSQL (production)
Testing	pytest 9, in-memory SQLite

Project Structure

Backend

File	Description
app.py	Flask application factory
config.py	Dev / Prod / Test configurations
models.py	SQLAlchemy models: User, LeaveBalance, LeaveRequest
requirements.txt	Python dependencies
routes/auth.py	Endpoints: register, login, get/update profile
routes/users.py	Endpoints: list users, leave balance, toggle admin
routes/leaves.py	Endpoints: submit, approve, reject, cancel requests
tests/conftest.py	Shared pytest fixtures (app + client)
tests/test_auth.py	12 tests - register, login, profile
tests/test_leaves.py	17 tests - submit, approve, reject, cancel
tests/test_users.py	11 tests - users, balances, admin toggle

Frontend

File	Description
src/api/axios.js	Axios instance with JWT header and 401 auto-logout
src/contexts/AuthContext.jsx	Auth state - login, logout, register, localStorage
src/components/Navbar.jsx	Top nav with links and logout button
src/pages/Login.jsx	Login form
src/pages/Register.jsx	Registration form with gender and department fields
src/pages/Dashboard.jsx	Leave balance cards and recent request summary
src/pages/ApplyLeave.jsx	Leave request form with live business-day counter

LeaveApp - Application Structure

src/pages/MyLeaves.jsx	Leave history with status filters and cancel button
src/pages/AdminPanel.jsx	3-tab admin: approve/reject requests, users, balances

API Endpoints

Auth (/api/auth)

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Login and receive JWT token
GET	/api/auth/profile	Get current user profile
PUT	/api/auth/profile	Update profile (name, department, gender)

Leaves (/api/leaves)

Method	Endpoint	Description
POST	/api/leaves/request	Submit a leave request
GET	/api/leaves/requests	List requests (own or all for admin)
GET	/api/leaves/requests/:id	Get a specific request
POST	/api/leaves/requests/:id/approve	Approve request (admin only)
POST	/api/leaves/requests/:id/reject	Reject request (admin only)
POST	/api/leaves/requests/:id/cancel	Cancel a request

Users (/api/users) - Admin only

Method	Endpoint	Description
GET	/api/users/	List all users
GET	/api/users/:id	Get a specific user
GET	/api/users/:id/leave-balance	Get user leave balances
POST	/api/users/:id/leave-balance	Set or update leave balance
POST	/api/users/:id/toggle-admin	Toggle admin role

Leave Types

Type	Default Days	Notes
Annual	20	General paid leave
Sick	10	Medical leave
Personal	5	Personal errands
Maternity	90	Female employees only
Bereavement	5	Loss of family member

LeaveApp - Application Structure

Test Coverage (40 tests, all passing)

File	Tests	Key Scenarios
test_auth.py	12	Register, login, wrong password, duplicate user, profile update
test_leaves.py	17	Submit, no balance, maternity restriction, approve, reject, cancel
test_users.py	11	List users, set/get balance, update balance, toggle admin on/off
Total	40	All passing - uses in-memory SQLite, no DB setup needed

Quick Start

Backend: `cd backend && pip install -r requirements.txt && python app.py`

Frontend: `cd frontend && npm install && npm run dev`

Tests: `cd backend && python -m pytest tests/ -v`

Backend: <http://localhost:5000> Frontend: <http://localhost:5173>

Default admin login - username: admin / password: admin